

PINO and PI-DeepONet baselines: what the implementation actually does

August 19, 2026

Summary of the two published label-free operator-learning baselines on branch `baselines-pino-deeponet`, read off the source rather than the papers. They exist so the comparison is against *published* physics-informed operator training and not only against our own $t = 0$ negative control. Code lives in `tensorpils/baselines/`; nothing in `losses.py`, `trainer.py` or `physics.py` is modified, and only `cli.py` gains dispatch.

The short answers

	PINO	PI-DeepONet
Architecture	FNO (ours, unchanged)	DeepONet (new, unstacked)
Derivatives	2nd-order central differences	autodiff through the trunk
Dirichlet BC	hard, boundary nodes zeroed	hard, $\sin(\pi x) \sin(\pi y)$
Residual form	strong	strong
Evaluated at	grid nodes, interior only	collocation points
Reduction	relative L^2 ratio	relative L^2 ratio

Does PINO just sit on top of the FNO? Yes. `ZeroBoundaryModel` wraps our `FNOModel` unchanged and zeroes its boundary nodes; the loss is swapped and nothing else. The only architectural addition in the whole branch is the DeepONet.

Boundary conditions

Both baselines impose $u = 0$ on $\partial\Omega$ **hard**, as a property of the *model* rather than of the loss, so training and evaluation see the same function. **They do so by different mechanisms, and deliberately so.**

PINO — zeroing the boundary nodes (default). The prediction is multiplied by a 0/1 mask that is zero on the outer ring of nodes. This is exactly the operation `losses.py` applies to our own arms via `apply_zero_boundary`, so PINO and the `galerkin/pls` arms now impose the Dirichlet condition identically and differ only in the residual — strong versus weak form, finite differences versus exact Q_1 assembly. A unit test asserts the two operations agree bit-for-bit.

This departs from the reference implementation, which multiplies by $\sin(\pi x) \sin(\pi y)$ (still available under `--pino.bc mollifier`). The reason is that the mollifier injects a smooth global decay profile into the model, and for the multi-frequency sine dataset used here that profile is close to

the solution’s own structure — information our arms never receive. It also does not generalise: zeroing boundary nodes works on any mesh, whereas a mollifier must be hand-built per geometry. *Expect this to cost PINO accuracy relative to the mollified version*; that is the price of making the comparison attributable.

Masking is bit-exact, so unlike the mollifier it needs no boundary snap: $\sin(\pi \cdot 1.0)$ evaluates to 8.7×10^{-8} in `float32`, not 0.

PI-DeepONet — the mollifier, for now. Unchanged from the reference: $\sin(\pi x)\sin(\pi y)$, evaluated at the *query* coordinates inside `forward_at`, so the autodiff Laplacian differentiates the mollified field as it must. The same objection applies here, and if the mollifier is reinstated it should be reinstated uniformly across all methods rather than for one arm.

This is an advantage the baselines have over our bare galerkin arm, which does not get a hard BC. It is deliberate: skipping it would be strawmanning. The soft BC penalty of the original PI-DeepONet (λ_{bc}) is therefore 0 by default, which removes a tuned hyperparameter rather than adding one.

How derivatives are computed

PINO: finite differences. The Laplacian comes from `neuralop.losses.differentiation.FiniteDiff` — the reference library’s own utility, second-order central in the interior, pure `torch` so it stays differentiable. Using the library’s function rather than a reimplementaion removes “you coded the baseline wrong” as a reading of any result.

The choice of scheme is not taste. The reference repo uses FFT differentiation for its *periodic* problems (Burgers, Navier–Stokes) and central differences for *Dirichlet* Darcy. Every PDE here is homogeneous Dirichlet on the unit square, so Darcy is the analogue and finite differences is the faithful choice.

Note `FiniteDiff` names axis -2 “`x`” while this repo names it `y`; the spacings are passed in its order, (h_y, h_x) .

PI-DeepONet: autodiff. The trunk consumes the coordinate $y = (x, y)$ directly, so the Laplacian is exact: one `grad` of u with respect to `coords`, then a second `grad` per dimension, with `create_graph=True` throughout so the residual remains differentiable in the parameters.

Coordinates are passed per sample, as `[B, Q, 2]` rather than `[Q, 2]`. With the shared form, `grad(u.sum(), coords)` returns $\sum_b \partial u_b / \partial y$ — the batch *sum* of gradients — and the per-sample residual cannot be recovered. The code raises if given the shared form.

The DeepONet architecture

Standard unstacked DeepONet,

$$\mathcal{G}_\theta(f)(y) = \frac{1}{\sqrt{p}} \sum_{k=1}^p b_k(f) t_k(y) + b_0,$$

with an MLP branch over the input function sampled on the sensor grid and an MLP trunk over the query coordinate. `neuralop` ships no DeepONet, so this is a direct implementation.

Component	Value
Basis functions p	128
Branch	MLP $4225 \rightarrow 256 \rightarrow 256 \rightarrow 256 \rightarrow 128$, tanh
Trunk	MLP $128 \rightarrow 256 \rightarrow 256 \rightarrow 256 \rightarrow 128$, tanh
Trunk input	64 random Fourier features (fixed, scale 2.0)
Sensors	the full 65^2 grid, flattened
Trainable parameters	1.44×10^6 (branch 1.25×10^6 , trunk 0.20×10^6)

Three implementation choices exist so that a poor result is attributable to the objective and not to a handicapped baseline:

- **Random Fourier features on the trunk.** A plain MLP has a strong spectral bias and these data are multi-frequency ($K = 4$ means modes up to 4×4); without them the DeepONet underfits for reasons unrelated to the loss. Standard practice in the PI-DeepONet literature. The matrix B is a fixed buffer, so it travels with the checkpoint.
- **$1/\sqrt{p}$ output scaling.** A dot product of two p -vectors grows like $\sqrt{p}$, so without this the output magnitude at initialisation depends on p (measured $5.5\times$ the target at $p = 32$, $12.4\times$ at $p = 128$). Starting an operator an order of magnitude too large costs the baseline real epochs.
- **Branch input scaling `f_scale`:** an MLP on a raw $O(10^2)$ input trains badly, while the FNO’s lifting layer absorbs the scale by itself.

It is also a drop-in for the FNO. `forward(f_grid)` returns $[B, 1, H, W]$, the FNOModel signature, so the DeepONet can be paired with *any* loss in `losses.py` through the unmodified `Poisson-Trainer`. This fills the “our loss, other architecture” cell for free. `forward_at(f_grid, coords)` is the second entry point, and is what the autodiff residual uses.

Things that are easy to miss

- **The reduction is a relative ratio, not our sum-over-nodes convention.** PINO reduces with $\text{mean}_b \|\Delta_h u - f\|_2 / \|f\|_2$. It is inherited, not a physics-informed design choice: this is `LpLoss.rel()` from the reference FNO/PINO codebase, the relative- L^2 reduction introduced as the *supervised* training loss in the original FNO paper and reused unchanged for the equation residual. It changes the gradient scale by *orders of magnitude* relative to our $\frac{1}{2} \|\cdot\|^2$ losses, which is why each baseline arm gets its own learning-rate sweep. Comparing at a shared learning rate would be meaningless. `--pino_reduction mse` puts them back on our footing when that is the question.
- **It is a ratio of norms, not of squared norms**, and that is not cosmetic: $\partial_r (\|r\|_2 / \|f\|_2) = r / (\|r\|_2 \|f\|_2)$ has norm $1/\|f\|_2$ for *every* $r \neq 0$, so the gradient magnitude does not vanish as the residual does (measured constant across six orders of magnitude in $\|r\|$, while $\frac{1}{2}\|r\|^2$ decays linearly). PINO’s objective is therefore not a smooth least-squares problem near its solution, its floor is set by the learning rate, and the Hessian analysis of Section 2.3—which presumes the squared form—does not transfer to it as written.
- **The residual is evaluated on interior points only**, boundary rows and columns sliced off, as the reference does.

- **Allen–Cahn: strong vs. weak, same time discretisation.** Where our FEM residual is $M \frac{u^{n+1}-u^n}{\tau} + a^2 A u^{n+1} - M \rho$, PINO drops the mass matrix and replaces $A \leftrightarrow -\Delta_h$. The time stepping (convex–concave or backward Euler) is identical, so the only thing under test is weak versus strong form.
- **Allen–Cahn reduces with mse, not the relative ratio:** the step residual has no right-hand side to normalise against.
- **Collocation defaults to the grid nodes** — for fairness (every arm then sees the same points), and for Allen–Cahn out of necessity: the residual needs u^n at each collocation point and the FEM Newton reference exists only at nodes. Off-node collocation would require interpolating the reference and would inject an error unrelated to the method. For Poisson the source is closed-form, so random collocation is also available.
- **detach_coupling mirrors our ACGalerkinLoss** and is set from `--bptt_mode` exactly as the FEM arm is, so the two differ in the residual and not in what the gradient flows through.
- **Scoring is unchanged:** both are evaluated with the repo’s FEM relative- L^2 metric and the eval-time boundary projection, so their numbers drop straight into the existing tables.

Invocation

```
--loss pino # FNO + FD strong form (boundary nodes zeroed)
--loss pideeponet --model deeponet # DeepONet + autodiff strong form
--model deeponet --loss pls # our loss, other architecture
```

Relevant flags: `--mollify {auto,on,off}` (auto = on for the baseline losses, off otherwise, so existing arms are unchanged), `--pino_bc {zero,mollifier}` (default zero), `--pi_n_colloc`, `--pino_reduction`, `--deeponet_p/_width/_depth`, `--trunk_fourier[_scale]`.